Laptop Distance and Posture Alert System Using Arduino UNO R4 Minima

Project Overview:

This project is a real-time laptop distance and posture alert system built using an Arduino UNO R4 Minima and an HC-SR04 ultrasonic sensor. The ultrasonic sensor continuously measures the distance between the user and the laptop, using the measured distance as an indication of whether the user is sitting at an appropriate distance from the screen.

A passive buzzer provides progressive audio feedback based on the user's distance. When the user moves closer to the laptop, the buzzer increases its beeping frequency, providing an increasingly urgent warning. When the user moves back to a safe distance, the buzzer automatically stops.

The project was developed as a hands-on application of ultrasonic distance measurement, real-time sensor monitoring, timing control, and human-machine feedback using Arduino.

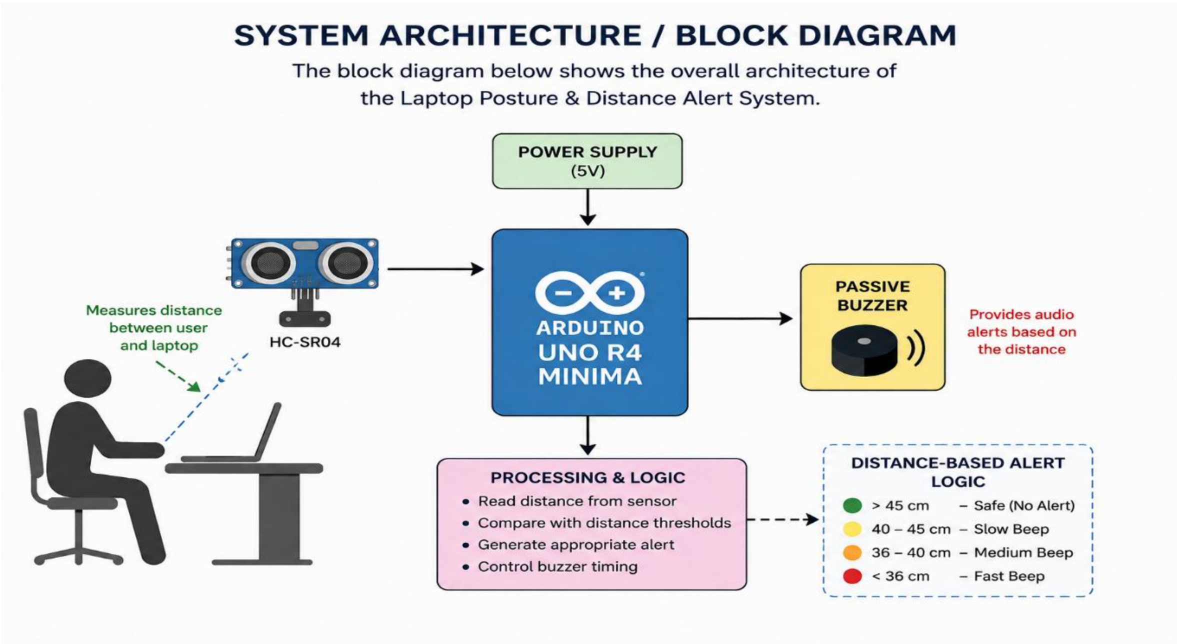
Objectives:

- To continuously measure the user's distance from the laptop using an HC-SR04 ultrasonic sensor.
- To use distance as a simple indicator of poor sitting posture or excessive proximity to the screen.
- To provide real-time audio feedback using a passive buzzer.
- To gradually increase the buzzer's beep rate as the user moves closer to the laptop.
- To automatically stop the warning when the user moves back to a safe distance.
- To gain practical experience with ultrasonic sensors, pulseIn(), tone(), timing functions, and real-time embedded-system logic.

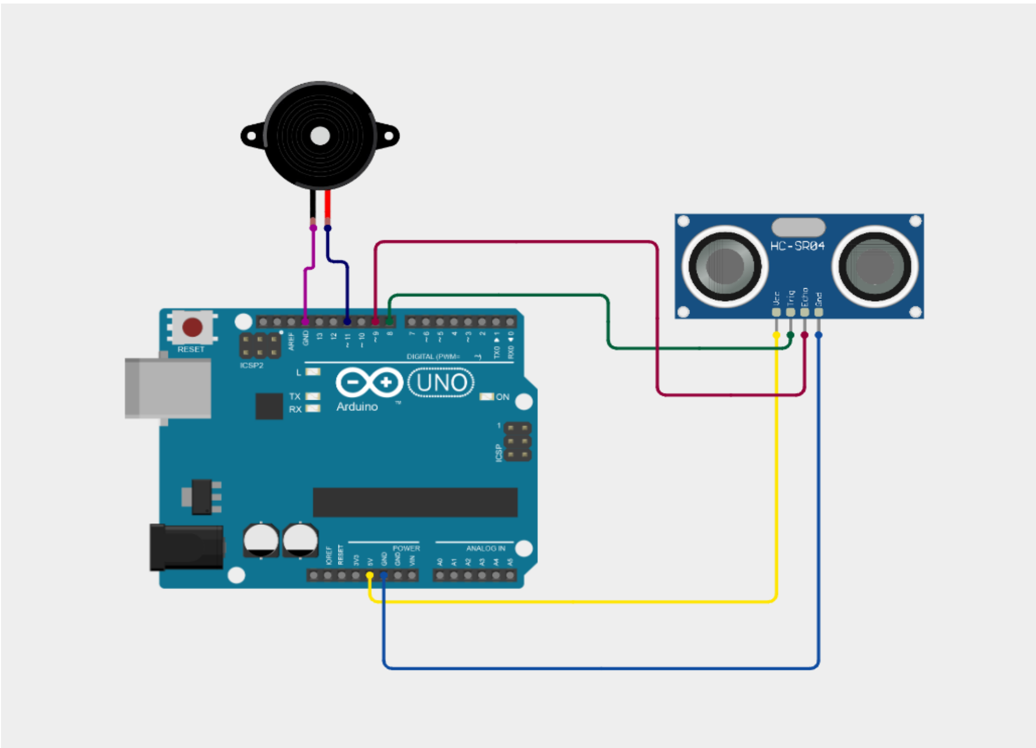
Components Used:

Component	Quantity	Purpose
Arduino UNO R4 Minima	1	Main controller
HC-SR04	1	Distance measurement
Passive Buzzer	1	Provides progressive audio warnings
Jumper Wires	—	Electrical connections

System Architecture / Block Diagram:



Circuit Diagram:



Prototype Images:

Figure 1. Hardware setup mounted on the laptop

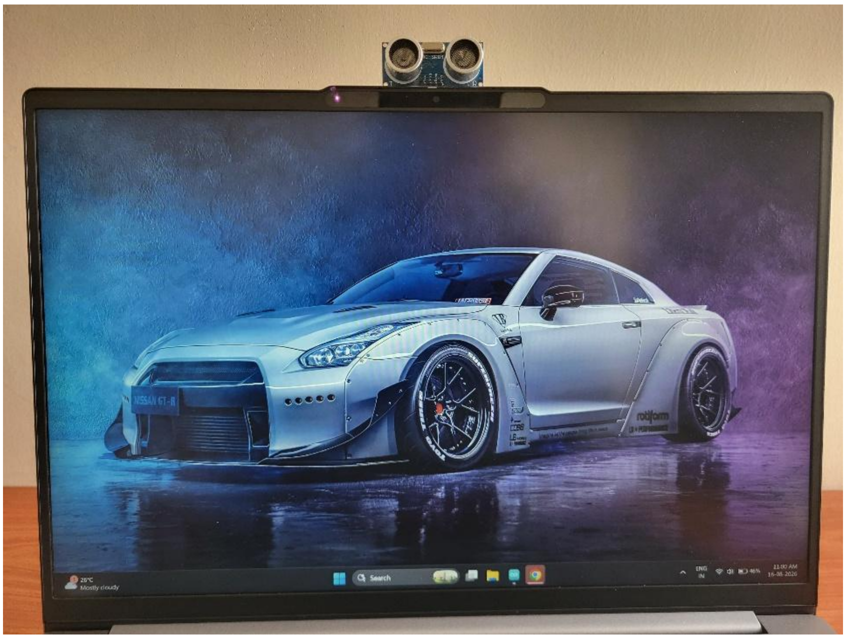


Figure 2. Ultrasonic sensor positioned for user-distance measurement

Pin Configuration:

Arduino Pin	Connected To
D8	HC-SR04 TRIG
D9	HC-SR04 ECHO
D11	Passive Buzzer
5V	HC-SR04 VCC
GND	HC-SR04 GND + Buzzer GND

Working Principle:

The HC-SR04 ultrasonic sensor continuously measures the distance between the user and the laptop. Arduino sends a short trigger pulse to the sensor and measures the time taken for the reflected ultrasonic wave to return.

The measured echo time is converted into distance using the speed of sound. Arduino then compares the measured distance with predefined distance ranges.

If the user is farther than 45 cm, the system remains silent. As the user moves closer, the passive buzzer produces warning beeps at increasing speeds. When the user moves back, the buzzer automatically adjusts or stops according to the new distance.

```
User
↓
HC-SR04
↓
Measure Distance
↓
Arduino UNO R4
↓
Compare Distance
↓
Buzzer Warning
```

Detection Logic:

The warning system uses different beep intervals depending on the user's distance from the laptop.

Distance	Warning	Buzzer Behavior
> 45 cm	Safe	Silent
40–45 cm	Initial warning	Slow beeping
36–40 cm	Increased warning	Faster beeping
≤36 cm	High warning	Very fast beeping

The tone frequency remains constant, while the time between beeps decreases as the user gets closer. This creates a progressive warning system where the increasing beep rate immediately indicates that the user is moving into an unsafe distance.

Software Design:

The software continuously measures the distance between the user and the laptop using the HC-SR04 ultrasonic sensor. The Arduino sends a trigger pulse and measures the duration of the returning echo using pulseIn().

The measured echo time is converted into distance using the speed of sound:

Distance = (Echo Time × 0.0343) / 2

The program then compares the measured distance with predefined thresholds and selects the appropriate buzzer interval.

The buzzer is controlled using tone() and noTone(). The millis() function is used to control the time between beeps without stopping the continuous distance measurements.

The main software flow is:

```
Measure Distance
↓
Calculate Distance
↓
Check Distance Range
↓
Select Beep Interval
↓
Control Passive Buzzer
↓
Measure Again
```

Limitations:

- The system uses distance as an indirect indication of posture; it does not directly analyze the user's body posture.
- Ultrasonic measurements can fluctuate depending on the user's position, movement, clothing, and surrounding objects.
- The HC-SR04 has a limited sensing range and field of view.
- The system currently provides only audio feedback.
- The buzzer's warning intensity is controlled by changing the beep rate, not the actual volume.
- The system does not distinguish between the user and other objects that may enter the sensor's detection area.

Future Improvements:

Possible improvements for future versions include:

- Add an OLED display to show the measured distance and current warning status.
- Add visual indicators such as LEDs.
- Add a calibration system for different users and desk setups.
- Implement more advanced posture detection using multiple sensors.
- Add data logging to track sitting distance over time.
- Add wireless connectivity for monitoring and notifications.
- Design a compact enclosure and a proper mounting mechanism for the sensor.
- Improve distance filtering to reduce fluctuations in ultrasonic measurements.

What I Learned:

Through this project, I gained practical experience in:

- Interfacing an HC-SR04 ultrasonic sensor with Arduino.
- Measuring distance using ultrasonic echo timing.
- Working with pulseIn() and sensor timeouts.
- Controlling a passive buzzer using tone() and noTone().
- Using millis() for non-blocking timing.
- Designing distance-based alert logic.
- Testing and tuning threshold values.
- Understanding the difference between blocking and non-blocking program logic.
- Integrating hardware and software to create a real-time interactive system.

Source Code:

The following Arduino program implements the complete distance-based alert system. It continuously measures the distance using the HC-SR04 ultrasonic sensor and dynamically changes the passive buzzer's beep interval based on the measured distance. Non-blocking timing using millis() allows the system to continue monitoring the sensor while controlling the buzzer.

Final Arduino Source Code:

```
int trigger = 8, echo = 9, buzzer = 11;
double distance = 0.0;
unsigned long lastBeep = 0, pingTT = 0;
bool buzzerState = false;

void setup() {
  pinMode(trigger, OUTPUT);
  pinMode(echo, INPUT);
  pinMode(buzzer, OUTPUT);

  Serial.begin(9600);
}

void loop() {
  digitalWrite(trigger, LOW);
  delayMicroseconds(5);
  digitalWrite(trigger, HIGH);
  delayMicroseconds(10);
  digitalWrite(trigger, LOW);
  pingTT = pulseIn(echo, HIGH, 15000);
  if(pingTT == 0)
  {
    noTone(buzzer);
    buzzerState = false;
    return;
  }
  distance = (0.0343 * pingTT) / 2.0;
  Serial.println(distance);
  if(distance>45)
  {
    noTone(buzzer);
    buzzerState = false;
  }
  else if(distance>40 && distance<=45)
    beep(1000);
  else if(distance>36 && distance<=40)
    beep(400);
  else
```

```
    beep(100);
}

void beep(unsigned long interval)
{
  if(millis() - lastBeep >= interval)
  {
    lastBeep = millis();
    if(buzzerState == false)
    {
      tone(buzzer, 3000);
      buzzerState = true;
    }
    else
    {
      noTone(buzzer);
      buzzerState = false;
    }
  }
}
```

References:

- Arduino documentation.
- HC-SR04 ultrasonic sensor documentation/datasheet.
- Arduino tone() and noTone() documentation.
- Arduino millis() documentation.
- The initial concept and sensor implementation were inspired by online Arduino tutorials. The warning logic and distance-dependent buzzer behavior were independently developed and modified for this project.